API перехвата ROM-команд MK61s mini

Версия документа: 23.07.2026

Назначение

ROM-hook API позволяет расширению эмулятора наблюдать или временно подменять любое командное слово одной из трех микросхем К145ИК:

- IK1302;
- IK1303;
- IK1306.

Можно одновременно зарегистрировать callback для нескольких команд, а также несколько callback для одной команды. Таблицы ROM при этом не изменяются.

API предназначен для низкоуровневых расширений ядра. Он работает с адресами командных слов ROM, а не с пользовательскими opcode программы МК-61.

Если требуется перехватить внешнюю команду вроде `D7` (КИП7) или `3B` (К СЧ), следует использовать command-hook API из `MK61s-mini-Command-Hooks.md`. Он одинаково обрабатывает клавиатуру и программу, поддерживает фазы BEFORE/AFTER и не требует знания внутреннего микрокода.

Модель выполнения

В каждой микросхеме находится таблица из 256 командных слов. Перед чтением очередного слова ядро вычисляет 8-битный адрес:

```
address = R[36] + 16 * R[39]
```

Точка регистрации задается парой:

```
RomChip + address
```

Если для этой пары есть hook, ядро синхронно вызывает зарегистрированные callback. После их завершения оно читает выбранное командное слово и продолжает обычный микрошаг.

Адрес ROM-команды не равен пользовательскому opcode. Например, opcode `0x3B` команды К СЧ проходит через несколько внутренних состояний, а нужная генератору точка имеет адрес `IK1306:A7`. При этом `A7` не служит признаком команды К СЧ: внешний opcode заранее распознается command-hook API, а `IK1306:A7` используется только для доступа к транзитному слову `xi`.

Публичные типы

Интерфейс объявлен в `code/mk61emu_core.h`:

```
enum class RomChip : u8 {
  IK1302,
  IK1303,
  IK1306
};

struct RomCommandHookContext {
  RomChip chip;
  u8 address;
  u8 replacement_address;
  u8* r;
  u8* st;
};

using RomCommandHook =
    void (*)(RomCommandHookContext& context, void* user_data);

using RomCommandHookHandle = u32;
```

`RomCommandHookHandle` идентифицирует одну регистрацию. Значение `INVALID_ROM_COMMAND_HOOK` равно нулю и означает ошибку регистрации.

Функции API

```
RomCommandHookHandle register_rom_command_hook(
    RomChip chip,
    u8 address,
    RomCommandHook callback,
    void* user_data = nullptr);

bool unregister_rom_command_hook(RomCommandHookHandle handle);

usize registered_rom_command_hook_count();

u32 rom_command_instruction(RomChip chip, u8 address);
```

`register_rom_command_hook()`

Создает независимую регистрацию для пары `chip:address`.

Функция возвращает `INVALID_ROM_COMMAND_HOOK`, если:

- `callback` равен `nullptr`;
- значение `chip` недопустимо;
- заполнены все пользовательские слоты;
- регистрация выполняется из уже работающего callback.

`user_data` передается callback без копирования. Вызывающая сторона отвечает за то, чтобы объект оставался действителен до удаления регистрации.

`unregister_rom_command_hook()`

Удаляет только регистрацию с переданным handle. Остальные hook, включая hook той же команды, продолжают работать.

В handle хранится поколение слота. Поэтому устаревший handle не может удалить более новую регистрацию, которая позднее заняла тот же слот.

Функция возвращает `false` для неизвестного, устаревшего или уже удаленного handle, а также при попытке удаления из выполняющегося callback.

Вспомогательные функции

`registered_rom_command_hook_count()` возвращает количество пользовательских регистраций. Встроенные hook ядра в это число не входят.

`rom_command_instruction()` возвращает исходное 32-битное слово из ROM по паре `chip:address`. Функция ничего не подменяет и полезна для диагностики и тестов.

Несколько регистраций

Одновременно доступны восемь пользовательских регистраций. Они могут перехватывать разные адреса любых микросхем:

```
RomCommandHookHandle hooks[] = {
  register_rom_command_hook(
      RomChip::IK1302, 0x12, callback_1302, data_1302),
  register_rom_command_hook(
      RomChip::IK1303, 0x34, callback_1303, data_1303),
  register_rom_command_hook(
      RomChip::IK1306, 0xA7, callback_1306, data_1306)
};
```

Один callback также можно зарегистрировать несколько раз с разными адресами или `user_data`. Каждая регистрация получает собственный handle.

Для удаления всего набора нужно удалить каждый handle отдельно:

```
for(RomCommandHookHandle handle : hooks)
  if(handle != INVALID_ROM_COMMAND_HOOK)
    unregister_rom_command_hook(handle);
```

Девятый слот зарезервирован для встроенного генератора К СЧ МК61s. Поэтому включение генератора не уменьшает публичную емкость из восьми регистраций.

Несколько callback одной команды

Пользовательские callback одной пары `chip:address` выполняются в порядке регистрации. Все они получают один общий объект контекста для текущего чтения.

```
RomCommandHookHandle first = register_rom_command_hook(
    RomChip::IK1306, 0x42, first_callback, first_data);

RomCommandHookHandle second = register_rom_command_hook(
    RomChip::IK1306, 0x42, second_callback, second_data);
```

Сначала будет вызван `first_callback`, затем `second_callback`. Удаление `first` не затрагивает `second`.

Встроенные hook ядра выполняются раньше пользовательских callback той же точки. Это существенно для `IK1306:A7`: пользовательский callback видит состояние уже после работы встроенного hook генератора.

Контекст callback

Поле	Назначение
`chip`	Микросхема, для которой выполняется текущее чтение
`address`	Исходный адрес ROM-команды, вызвавший цепочку callback
`replacement_address`	Адрес слова, которое ядро реально прочитает после callback
`r`	Указатель на 42 тетрады массива R выбранной микросхемы
`st`	Указатель на 42 тетрады массива ST выбранной микросхемы

Наблюдение

Callback может только наблюдать выполнение:

```
void observe(RomCommandHookContext& context, void* user_data) {
  Counter* counter = (Counter*) user_data;
  counter->calls++;
  counter->last_address = context.address;
}
```

Подмена командного слова

Для текущего чтения можно выбрать другой адрес той же ROM:

```
void replace(RomCommandHookContext& context, void*) {
  context.replacement_address = 0x42;
}
```

Изначально `replacement_address` равен `address`. Следующий callback исходной цепочки видит результат предыдущего и может оставить или изменить его снова. Поле `address` всегда содержит исходный адрес.

Подмена действует только для текущего чтения. Она не изменяет:

- таблицу ROM;

- регистры `R[36]` и `R[39]`;
- следующие чтения той же команды.

После подмены диспетчер не запускает новую цепочку для `replacement_address`. Выполняются только callback, зарегистрированные для исходной пары `chip:address`.

Доступ к R и ST

Указатели `r` и `st` действуют только во время callback. Их нельзя сохранять и использовать после возврата из функции.

Массив ST является транзитным рабочим состоянием микросхемы. Один индекс не соответствует постоянному логическому регистру: смысл данных зависит от микротакта и текущей ROM-команды. Расширение должно заранее знать точную точку перехвата и раскладку данных именно в ней.

Неверная запись может повредить BCD-формат или внутреннее состояние эмулятора. Если расширению не требуется изменение R или ST, безопаснее считать эти указатели доступными только для чтения.

Ограничения жизненного цикла

Регистрировать или удалять hook из выполняющегося callback нельзя. Такие операции отклоняются, чтобы порядок обхода оставался детерминированным.

Callback выполняется синхронно внутри цикла эмулятора и не должен запускать рекурсивное выполнение ядра. Долгая работа callback напрямую увеличивает время каждого перехваченного чтения ROM-команды.

API не владеет `user_data`, не вызывает деструкторы и не выделяет динамическую память.

Быстрый путь

Для трех ROM используется битовая карта адресов. Если для текущей пары `chip:address` ничего не зарегистрировано, ядро сразу читает ROM и не обходит реестр callback.

В прошивках STM32F401/F411 карта занимает 96 байт, а каждый её бит читается и изменяется через SRAM bit-band alias Cortex-M4. Alias-база вычисляется один раз на цикл эмулятора и используется всеми тремя микросхемами. В результате неперехваченная ROM-команда проверяется одной индексированной загрузкой `LDR.W` и переходом, без сдвига, маски и `LDRB`. Регистрация и удаление hook атомарно записывают отдельное alias-слово. Ручных вставок ассемблера нет: инструкции генерируются из C++-доступа через `volatile`-указатель.

Полный обход фиксированного реестра выполняется только для перехваченных адресов. Это сохраняет небольшой и предсказуемый расход памяти прошивки.

Как через API сделан К СЧ MK61s

Расширенный генератор использует оба уровня перехвата. При включении режима К СЧ MK61s функция `configure_random_seed()` регистрирует два внутренних callback:

```
command hook: opcode = 0x3B, phase = BEFORE_EXECUTE
ROM hook:     chip = IK1306, address = 0xA7
```

Command hook надежно распознает именно пользовательскую команду К СЧ из клавиатуры или программы и устанавливает одноразовый флаг. Он работает после публичной цепочки замен, поэтому учитывает итоговый `replacement_opcode`.

ROM hook срабатывает только при установленном флаге. Точка `IK1306:A7` соответствует единственному проверенному моменту, когда внутренний ROM-переход готовится прочитать прежнее логическое слово `xi`. Callback получает новое семизначное состояние, записывает его через `context.st`, снимает одноразовый флаг и оставляет `replacement_address` без изменения. Штатный микрокод сразу потребляет новое `xi` и помещает результат в X.

ROM не патчится, а seed не хранится заранее в ST. При выборе режима К СЧ MK61 оба внутренних hook удаляются, и эмулятор выполняет исторический алгоритм без внешнего вмешательства.

Контракт внешнего API описан в `MK61s-mini-Command-Hooks.md`, а источник начального материала и формат `xi-` в `MK61s-mini-Random.md`.

Проверки

Тесты в `tests/mk_math_self_test.cpp` проверяют одновременные hook разных ROM-адресов, порядок цепочки и `replacement_address`; независимое удаление и устаревшие handle; емкость из восьми регистраций и отдельный резерв ROM-hook ГСЧ. Взаимодействие внешних opcode с ГСЧ проверяется тестами command-hook API.